Midterms Pt 1 report: workspace MCP server

ARMAMENTO, Roderick Jr. T.
BSCS-3B

Repository: https://github.com/nakateruE/mcp-workspace-server

1. Stack and why

Layer	Choice	Why	Alternative considered
Runtime + SDK	Node.js + TypeScript MCP SDK	Official SDK, typed tool schemas, one language for server, tests and smoke script	Python + FastMCP
Transport	Streamable HTTP	Stateless request/response, works through a tunnel to a real remote client	stdio (local only)
Validation	zod	Every tool input is validated before any file is touched; schemas double as tool descriptions for the agent	pydantic
Exposure	ngrok static domain	Free stable HTTPS URL, so the client connector never has to be reconfigured	Cloudflare Tunnel, small VPS
Process manager	pm2	Restarts the server and tunnel if they crash; one JSON log line per tool call. pm2 startup is not supported on	systemd, Docker

		Windows, so after a PC restart I run pm2 resurrect.reboot, one JSON log line per tool call	
Version control	git on the host	Every session ends in a commit, so the workspace history is the audit trail	none
Testing	node:test, smoke script with a real MCP client, MCP Inspector	Unit tests for the sandbox, a scripted client that exercises every tool and tries escapes, a GUI for manual checks	Scripted raw JSON-RPC calls

2. Tool design choices

The server has 10 small single-purpose tools: `list_files`, `read_file`, `write_file`, `append_file`, `str_replace`, `delete_file`, `search`, `git_status`, `git_commit`, `git_log`. Small tools keep each call cheap, make the agent's intent obvious in the logs, and let each tool carry its own guard instead of one "do anything" tool that would need every guard at once.

- **`str_replace` needs exactly one match.** A wrong or ambiguous edit fails loudly instead of silently changing the wrong place, and sending only the changed text is far cheaper than rewriting a file.
- **`read_file` supports line ranges and returns a version hash.** The agent can read only the part it needs, and `if_version` on `write_file` and `str_replace` rejects a write if the file changed since it was read (conflict detection between sessions).
- **`append_file` exists and `PLAN_LOG.md` is append-only.** The history of what happened cannot be erased or rewritten by an agent.
- **Protocol files cannot be deleted.** The handoff depends on them.
- **The server is stateless.** There is no session to lose when pm2 restarts it.

3. Handoff protocol

The workspace is the only memory shared between sessions. `AGENT_README.md` tells each agent the routine: read `PROJECT_STATE.md` (goal, status, next step), read `DECISIONS.md` (settled choices, so they are not re-decided), then read only the newest file in `CHECKPOINTS/`. During work, small edits use `str_replace` and each step is appended to `PLAN_LOG.md`. At

the end the agent writes the next `CHECKPOINTS/cp-NN.md`, updates `PROJECT_STATE.md` and commits.

The demo task was a small Node command-line todo app (add, list, done, JSON storage, README, 3 tests). Session 1 started from a clean workspace with the goal in its first message. It filled in PROJECT_STATE.md, recorded eight design decisions in DECISIONS.md, wrote the add and list code (src/store.js, src/commands.js, bin/todo.js, package.json), wrote CHECKPOINTS/cp-01.md stating that the code was untested, and committed (2661633).

Session 2 was a brand-new chat that started with only the instruction "read AGENT_README.md and continue the project from where the last session stopped." Nothing was re-explained. The agent read the protocol files and the existing code, then did exactly the recorded next step: it added done <id>, wrote 3 tests and the README, kept to the recorded decisions (plain Node with no dependencies, node:test) without asking, wrote cp-02.md, updated PROJECT_STATE.md and committed (a571fd1). After the user ran npm test on the real files (3 pass, 0 fail), a short follow-up updated PROJECT_STATE.md and PLAN_LOG.md to mark the project verified and committed (db3d052).

4. Security tradeoffs

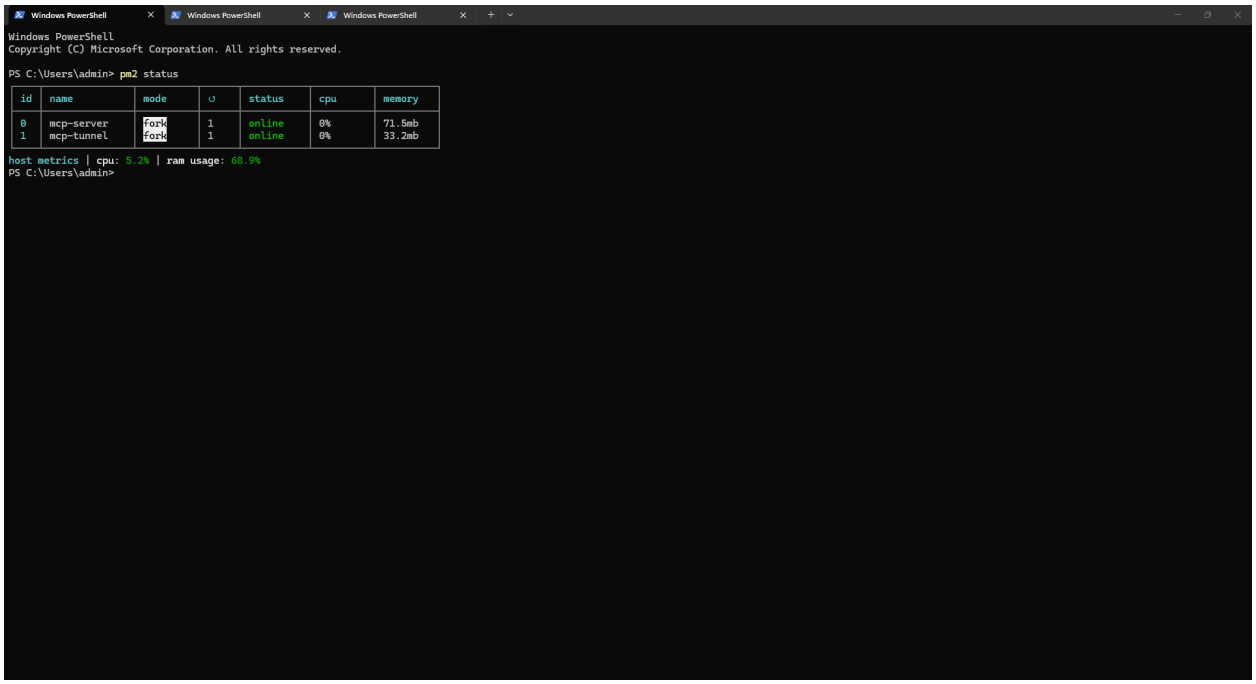
- **Sandbox.** Every path is resolved against the workspace root and checked for containment using `root + separator`, so a sibling directory like `workspace-evil` does not pass a naive prefix check. Absolute paths and `..` escapes are rejected, NUL bytes are rejected, `.git` is blocked, and symlinks are refused. The tradeoff is that symlinks are unsupported.
- **No shell.** Git runs through `execFile` with an argument array, so there is no shell to inject into, and commit messages are length-capped.
- **Guards on writes.** 1 MB write cap, append-only `PLAN_LOG.md`, protected protocol files, `str_replace` needs exactly one match, optional `if_version`.
- **Auth.** A static token, accepted as an `Authorization: Bearer` header or as a secret in the URL path for clients that cannot send headers. This does not protect against a leaked URL (anyone holding it can edit the workspace), and traffic passes through ngrok, a third party. The token should be rotated after the demo.
- **Not covered, and next steps.** Rate limiting, per-user auth (OAuth), and an audit log beyond pm2's logs.
- **Windows-specific hardening.** The first version of the sandbox compared names case-sensitively, so plan_log.md or .GIT could slip past the append-only and .git guards on Windows. The checks now ignore case and trailing dots/spaces, and paths containing ":" are rejected on Windows to block NTFS alternate data streams. The smoke test includes case-trick checks for both.

5. Results

- **Transcript summary.** The pm2 log shows 40 workspace tool calls in total: Session 1 took 17 calls in about a minute, Session 2 took 20 calls in about a minute and a half, and the follow-up after the user's npm test took 3. Session 2's 20 calls were 8 read_file and 1 list_files to get oriented and read the existing code, 5 str_replace edits, 3 write_file (tests, README, checkpoint), 2 append_file (plan log) and 1 git_commit. Nothing had to be re-explained, and the agent used if_version on its first str_replace in Session 1.
- **What worked.** Resuming from the checkpoint with no re-explaining; small targeted edits (8 str_replace calls across both sessions) instead of rewriting files; the user's npm test run confirmed 3 tests passing and 0 failing on the real workspace files.
- **What went wrong (session agent).** The workspace connector has no command execution, so the agent could not run the code on the real workspace files. In Session 1 it said so and marked the code untested. In Session 2 it copied the file contents into its own separate sandbox and ran npm test there (3 of 3), and said that a run on the real files would confirm it, so the user's run on Windows was the first one on the real files. The agent also suggested cd todo && npm test, which fails in older Windows PowerShell because && is not supported; the agent could not see which shell the user had.
- **What went wrong (setup help, separate from the session agent).** The chat assistant that helped with setup first advised pushing from the workspace folder, which was wrong because the workspace is a separate nested git repo.
- **What I would improve.** Add a tool that can run a fixed test command inside the sandbox, stamp log lines with the time server-side, and add OAuth and rate limiting.

6. Appendices

- **processes are online**



- **the tool-call log**

```
# PLAN_LOG (append-only, use append_file)
- [2026-10-17 00:00] read AGENT_README.md, PROJECT_STATE.md, DECISIONS.md; no prior checkpoints: ok
- [2026-10-17 00:01] set goal in PROJECT_STATE.md: done
- [2026-10-17 00:02] recorded design choices in DECISIONS.md: done
- [2026-10-17 00:03] wrote todo/src/store.js (JSON load/save): done, untested
- [2026-10-17 00:04] wrote todo/src/commands.js (add, list, formatList): done, untested
- [2026-10-17 00:05] wrote todo/bin/todo.js (CLI entry for add/list) and todo/package.json: done, untested
- [2026-10-17 00:06] wrote CHECKPOINTS/cp-01.md and updated PROJECT_STATE.md: done
- [2026-10-17] step: read README/STATE/DECISIONS/cp-01 and the todo/ sources
- [2026-10-17] step: implemented `done <id>` in commands.js and bin/todo.js (usage text updated)
- [2026-10-17] step: wrote todo/README.md and todo/test/todo.test.js (3 tests)
- [2026-10-17] step: mirrored todo/ into a sandbox copy (workspace cannot execute code); smoke test OK, `npm test` 3/3 pass
- [2026-10-17] step: user ran `npm test` in the real workspace on Windows - 3/3 pass; project goal verified
```

- **public HTTPS works**



- **pm2 restart mcp-tunnel**

```
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server> pm2 restart mcp-tunnel
Use --update-env to update environment variables
[PM2] Applying action restartProcessId on app [mcp-tunnel](ids: [ 1 ])
[PM2] [mcp-tunnel](1) ✓
```

id	name	mode	↺	status	cpu	memory
0	mcp-server	fork	1	online	0%	76.9mb
1	mcp-tunnel	fork	1	online	0%	31.1mb

```
host metrics | cpu: 8.4% | ram usage: 67.1%
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server> curl.exe https://resupply-glamour-snipping.n
grok-free.dev/health
{"ok":true}
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server> $env:MCP_URL="https://resupply-glamour-snipp
ing.ngrok-free.dev/mcp"; npm run smoke

> mcp-workspace-server@1.0.0 smoke
> tsx scripts/smoke.ts

PASS 10 tools: list_files, read_file, write_file, append_file, str_replace, delete_file, search, git_status, git_commit,
 git_log
PASS list_files
PASS write_file
PASS read_file
PASS str_replace
PASS str_replace rejects 0 matches
PASS if_version conflict detected
PASS search
PASS git_status
PASS git_log
PASS path escapes rejected
PASS PLAN_LOG.md overwrite blocked
PASS protected file delete blocked
PASS append-only guard is case-insensitive
PASS .GIT (different case) blocked
PASS oversize write blocked
PASS delete_file

ALL SMOKE CHECKS PASSED
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server>
```

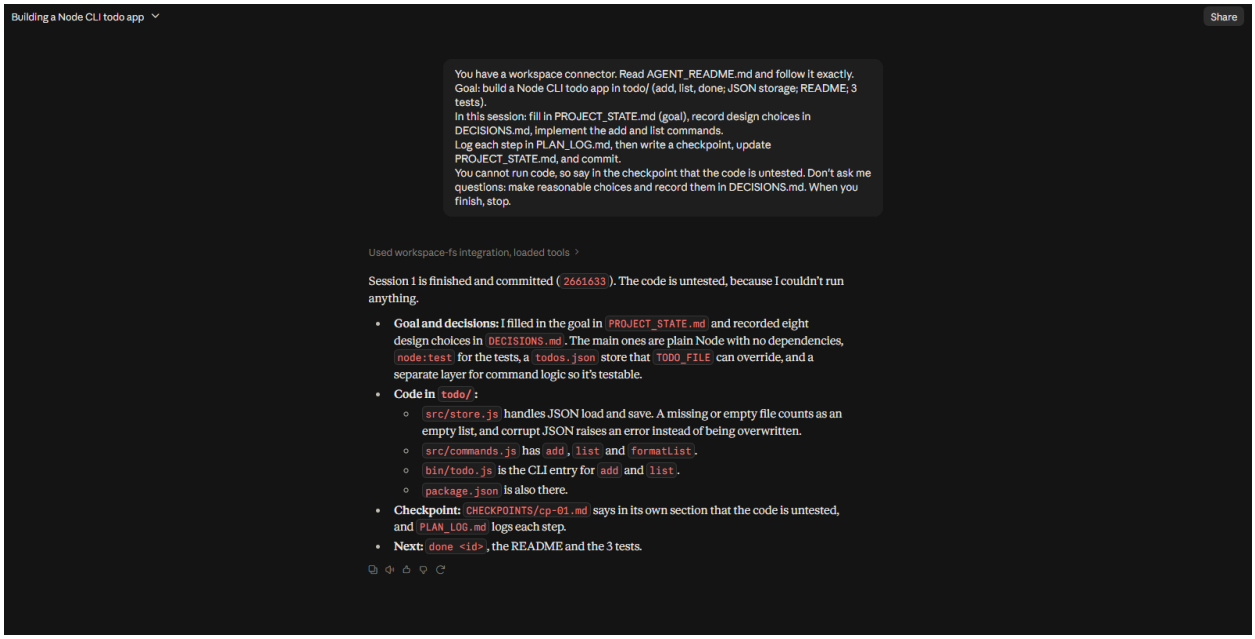
- **NPM test**

```
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server> npm test

> mcp-workspace-server@1.0.0 test
> tsx --test test/*.test.ts

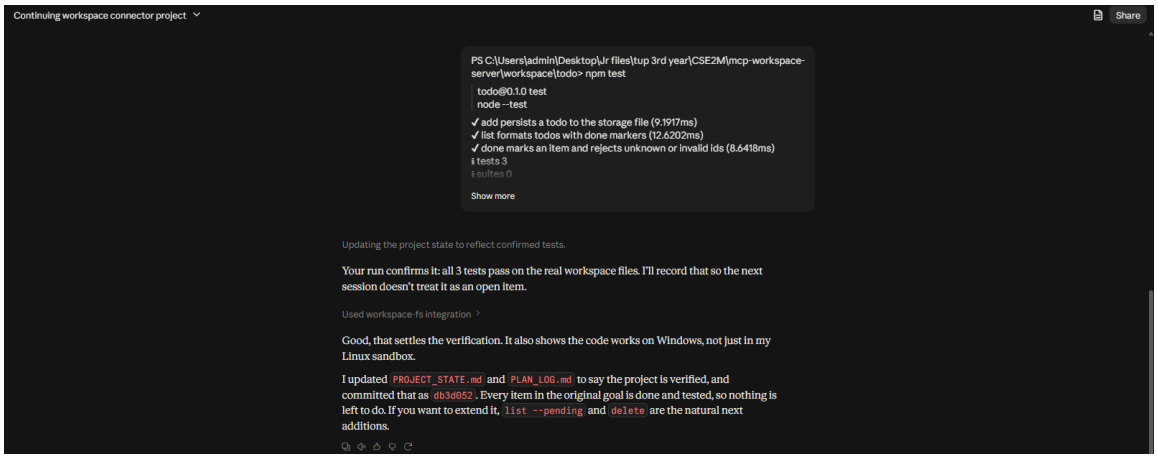
✔allows normal paths (13.4298ms)
✔rejects ../ traversal (6.1813ms)
✔rejects absolute paths outside root (5.2072ms)
✔rejects sibling directory with the same prefix (6.0714ms)
✔rejects symlink pointing outside (8.1347ms)
✔rejects dangling symlink out of root (write-through attack) (6.5249ms)
✔rejects .git access and NUL bytes and empty path (5.2121ms)
ℹtests 7
ℹsuites 0
ℹpass 7
ℹfail 0
ℹcancelled 0
ℹskipped 0
ℹtodo 0
ℹduration_ms 234.5877
```

- **Chat 1 Test**



- **Chat 2 Test**


You have a workspace connector. Read AGENT_README.md and continue the project from where the last session stopped.
Used workspace-fs integration, ran 2 commands, loaded tools
I finished the todo CLI from where the last session stopped. It now has add, list and done <id>, JSON storage, a README, and 3 tests.
Testing: The workspace connector can't run code, so I copied the same file contents into a sandbox (Node 22) and ran them there. A manual add/list/done run worked, and npm test passed 3 of 3. A missing id, an unknown id and an unknown command each print an error and exit 1. I haven't run it on the real workspace files, so cd todo && npm test on your side would confirm.
done: It marks the todo as done, and running it again on an already-done todo does nothing. A missing or non-numeric id gives Invalid id, and an unknown id gives No todo with id N.
Files: I wrote todo/README.md and todo/test/todo.test.js. The tests use temp files and cover add, the list formatting, and done with bad ids.
Bookkeeping: I logged the steps in PLAN_LOG.md and DECISIONS.md, wrote CHECKPOINTS/cp-02.md, updated PROJECT_STATE.md, and committed (a57ifd1).
Nothing is blocking. Optional extras would be a list --pending filter or a delete command.
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server\workspace\todo> npm test
todo@0.1.0 test
node --test
✓ add persists a todo to the storage file (9.1917ms)
✓ list formats todos with done markers (12.6202ms)
✓ done marks an iten rejects unknown or invalid ids (8.6418ms)
ℹ tests 3


Continuing workspace connector project
Share
PS C:\Users\admin\Desktop\Jr files\tup 3rd year\CSE2M\mcp-workspace-server\workspace\todo> npm test
todo@0.1.0 test
node --test
✓ add persists a todo to the storage file (9.1917ms)
✓ list formats todos with done markers (12.6202ms)
✓ done marks an item and rejects unknown or invalid ids (8.6418ms)
ℹ tests 3
ℹ suites 0
Show more
Updating the project state to reflect confirmed tests.
Your run confirms it: all 3 tests pass on the real workspace files. I'll record that so the next session doesn't treat it as an open item.
Used workspace-fs integration
Good, that settles the verification. It also shows the code works on Windows, not just in my Linux sandbox.
I updated PROJECT_STATE.md and PLAN_LOG.md to say the project is verified, and committed that as db3d052. Every item in the original goal is done and tested, so nothing is left to do. If you want to extend it, list --pending and delete are the natural next additions.